WMTM components guide: what each part does, why it exists, and how it works

For Ben. Written by ProtomegaTron for hive task t-2026-11-13-wmtm-components-guide, 2026-11-13. Base: wmtm `main` at e59ab57. The companion overview (WMTM_OVERVIEW.md, same folder) is the short, evidence-dense version; this guide is the long, explanatory one.

How to read this guide

You designed WMTM (the design doc of 5 September 2026) but have not read the code that grew from it. This guide walks through the system one component at a time, in the order a note travels through it. Every component section has the same seven parts:

1. **Purpose** - what it is for and where it sits in the flow.
2. **Inputs and outputs** - what it reads and what it produces.
3. **How it works** - the mechanism, step by step.
4. **Worked example** - the component applied to the running example (synthetic data only).
5. **Differences from the design doc, and why.**
6. **Status** - what is on main, what is switched on, what was measured.
7. **Limits** - what it does not do, or does badly.

Where one of the seven does not apply, the section says so. The prose avoids task ids, commit hashes, file paths and flag names. Those are collected at the end of each section under **Sources**, numbered [1], [2], ... so you can check any claim. Terms are defined once, in the glossary (section 2), and used consistently after that. A statement marked **[unverified]** is one I took from a report or the task record and could not re-check myself.

No real note text appears anywhere. The running example and every smaller example are invented.

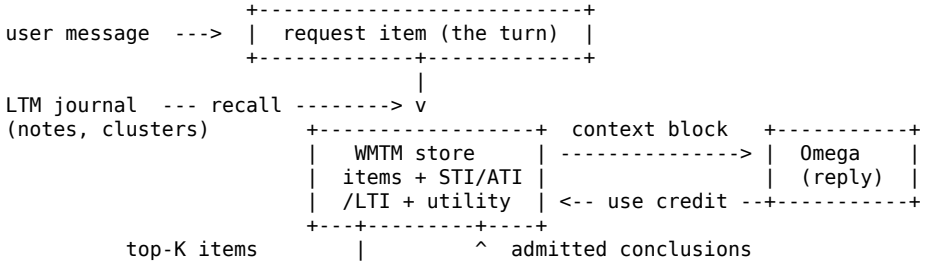
1. The big picture

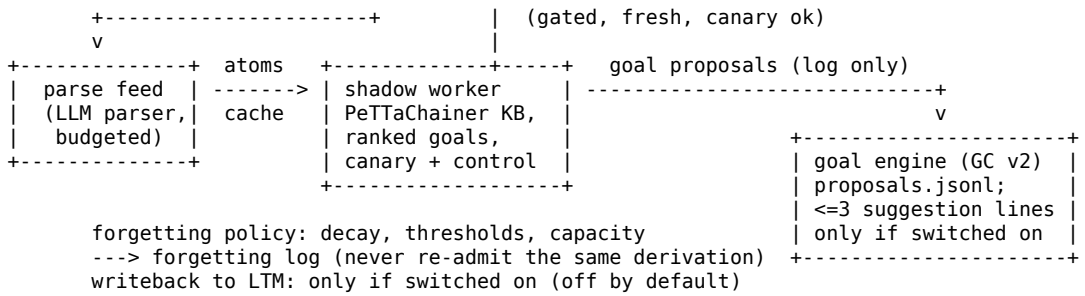
WMTM is a *working memory* for the Omega bot. The bot's long-term memory (LTM) is a large, append-only journal of notes. Most of it is irrelevant to any given conversation. WMTM holds a small, changing selection of it - at most a few hundred items - plus conclusions derived from those items, each with an attention value that rises when the item is used and decays when it is not. Every turn, WMTM shows the bot a short "context block" drawn from that selection. Items that stop being useful are forgotten, and the forgetting is logged so the same conclusion is not re-derived and re-forgotten in a loop.

Around that core, three background machines run in separate, low-priority processes so the bot never waits on them:

- a **parse feed** that turns notes into logical atoms using Man Hin's semantic parser (an LLM prompt);
- a **reasoner** (PeTTaChainer) that chains those atoms into new conclusions, some of which WMTM admits;
- a **goal engine** (GoalChainer v2) that looks for open questions and unkept promises and proposes what the bot could do about them. Today it only writes proposals to a log.

The data flow, one turn at a time:





Two things in this picture are easy to miss. First, the reasoner and the parser never run inside the turn: the turn reads whatever their *last* completed run left behind, and only if that run passed its self-checks. Second, almost every background feature is switched off unless the bot's flags file switches it on; section 13 explains the switches and what the live bot is believed to run.

2. Glossary

- **Item** - one entry in the WMTM store: a recalled note, a derived conclusion, or the current request.
- **Source type** - what kind of item it is: *recalled* (from LTM), *derived* (a reasoner conclusion), *user* (the current request).
- **STI / ATI / LTI** - short-, medium- and long-term importance, the three attention numbers each item carries. STI is "how relevant right now"; it decays fastest.
- **Tick / cycle** - one round of WMTM bookkeeping, normally once per bot turn.
- **Utility** - a running score of how often an item was actually used, minus a small penalty for idle turns.
- **Context block** - the short text WMTM hands the bot each turn, listing the most relevant items.
- **Recall** - pulling notes from LTM into the store, by keyword match and by following links between notes.
- **Cluster** - LTM's unit of storage: a group of related notes with links (evidence-for, promotes-from, supersedes).
- **Atom** - a logical statement in PLN form, such as (Member mira falconer) or (Inheritance falconer early_riser).
- **Member / Inheritance / Implication** - "this individual is in that class" / "every member of this class is in that class" / "if this holds, that holds".
- **Truth value (stv)** - a pair (strength, confidence): how true, and how much evidence stands behind it.
- **Semantic parser (sph)** - Man Hin's LLM prompt that turns a note into atoms.
- **Lowering** - wmtm's step that rewrites parser atoms into the forms PeTTaChainer accepts.
- **PeTTaChainer** - the reasoner: a backward chainer for PLN written in MeTTa, running on PeTTa (a MeTTa implementation on SWI-Prolog).
- **Goal (reasoner sense)** - a statement WMTM asks PeTTaChainer to prove, e.g. (Member mira lantern_keeper).
- **Goal (goal-engine sense)** - something the bot might do: ask a clarifying question, remind someone, etc. The two senses are different; the guide always says which one.
- **Shadow worker** - the background process that builds a small knowledge base and runs PeTTaChainer.
- **admit file** - the shadow worker's output: only the conclusions that passed every check.
- **Canary / negative control** - a made-up problem whose answer is known (must be proved) and one that must *not* be proved. If either comes out wrong, the whole run is discarded.
- **Trivial conclusion / restatement** - a conclusion that only repeats an asserted fact in another form.
- **Skolem (sk_ constant)** - a placeholder name the parser invents for "someone" or "something".

- **Frozen parse** - a stored parser output reused across experiments, so both arms see identical atoms.
- **A/B replay** - running the same notes through two configurations (A = current, B = candidate) and comparing.
- **Count rule** - andy's acceptance rule for A/B: B may not have more wrong conclusions, nor fewer correct ones.
- **Flags file** - a JSON file in the bot's memory folder that switches features on and off without a restart.

3. The running example, end to end

Every component section returns to this one invented scenario. Here it is in one pass, so you can see the whole journey before the details. All names, notes and numbers are made up.

The notes. Two older notes sit in LTM:

- Note A: "Mira trains the falcons at the north tower."
- Note B: "Everyone who trains falcons at the tower keeps the lantern log."

The turn. A user asks the bot: "Who keeps the lantern log this week?"

1. **Request item.** The question itself becomes an item of source type *user*, so later steps know what the current request is about (section 5).
2. **Recall.** The words "lantern" and "log" match Note B by keyword. Note A shares no keyword with the question, but it is linked to Note B in LTM, so graph recall can still pull it in (section 8). Both enter the store with fresh STI.
3. **Context block.** WMTM ranks the resident items by relevance to the request and by STI and shows the best few to the bot (section 8). The bot can already answer from Note B plus Note A, but it has to do the joining itself.
4. **Parse feed.** Both notes are queued for the semantic parser, which runs in the background within an hourly budget (sections 4 and 14). Its output, after lowering and the quality gate, is:
   - from A: (Member mira falcon_trainer)
   - from B: (Inheritance falcon_trainer lantern_keeper)
5. **Reasoner.** On its next run the shadow worker loads these atoms into PeTTaChainer, ranks candidate goals by how well they match the request, and asks, among others, (Member mira lantern_keeper). PeTTaChainer proves it by one Member-then-Inheritance step, with a truth value computed from the two premises (section 9).
6. **Admission.** The run's canary was proved and its negative control was not, the result is fresh, both source notes are still in the store, and the proof is a real derivation rather than a restatement. So on the next turn WMTM admits "mira is a lantern_keeper" as a *derived* item, recording its sources (A, B), its truth value and its proof (sections 6 and 7). If the reasoner had instead "proved" (Member mira falcon_trainer), which only repeats Note A, the trivial filter would have refused it (section 7).
7. **Goal engine.** Suppose Note B also said "the log may move to the gatehouse; Mira to confirm". The goal engine, if switched on, sees an open question addressed to Mira and proposes a ClarifyQuestion. It writes that proposal to its log. Only with a second switch on, and only when Mira herself is the one talking to the bot, would it show up as a line marked "suggestions (not instructions)" (section 10).
8. **Use and decay.** If the bot's reply uses the derived item, its utility rises. If the conversation moves on, its STI decays every tick (section 7).
9. **Writeback or not.** The derived item could be written back to LTM as a permanent belief if it stayed useful for long enough, but that path is off by default, so in practice it is not (section 7).
10. **Forgetting.** A few turns later its STI falls below the floor for derived items and it is evicted. The forgetting log records the *derivation* (the conclusion together with its sources), so the next reasoner run does not re-admit the same conclusion from the same notes. If a new note later gives fresh support, that is a different derivation and can be admitted (section 7).

The rest of the guide takes each numbered step apart.

4. Note intake and the semantic parser

4.1 Purpose

The reasoner cannot read English. Before PeTTaChainer can chain anything, each note has to be turned into logical atoms. This component does that translation. It sits between the store and the reasoner. It takes the items that currently have the most attention, sends their text to Man Hin's semantic parser, cleans up what comes back, and keeps the result in a cache that the reasoner reads later. It never runs inside a turn. A turn only *queues* work, and a background worker does the parsing when it has budget.

4.2 Inputs and outputs

- **In:** the text of the top-K items in the store (by attention), each identified by a hash of its content.
- **Out:** for each parsed item, a list of atoms that PeTTaChainer has accepted (Member, Inheritance, Implication, Evaluation and similar statements, each with a parse confidence). These go into a parse cache. A log records every parse attempt: which prompt was used, how many tokens it cost, and whether it succeeded.
- **Side effect:** one row in the shared parse-budget ledger per call to the LLM (section 14).

4.3 How it works

1. **Queue.** On each cycle, WMTM takes the top-K items and puts their content hashes in a queue. If the top-K set has not changed since the last cycle, nothing is queued. Content that was already parsed is never queued again, so a note costs at most one parse in its lifetime (unless someone deletes its cache entry on purpose).
2. **Launch.** A detached worker process starts if none is running. It runs at the lowest CPU priority, so the bot never waits for it.
3. **Prompt.** The worker sends each note to the bot's own LLM gateway with Man Hin's prompt. The prompt is pinned to a fixed version, and the worker checks its checksum before every run. A short instruction preamble written by wmtm is put in front of the pinned prompt; the pin check runs on the unedited file. Notes longer than a size limit are skipped, and a failed call is retried once.
4. **Load check.** Every atom the parser returns is loaded into PeTTaChainer on its own. Atoms that PeTTaChainer rejects (bad syntax, an unknown form) are dropped. Only atoms that load are cached.
5. **Lowering.** Some forms the parser emits are valid PLN but are refused by PeTTaChainer, or are useless to it. wmtm rewrites them before loading:
   - *Narrowed classes.* The parser often describes a class by narrowing a broader one, such as "trainer, of falcons". As a standalone class, wmtm gives such a narrowed class a fixed, generated name (e.g. `falcon_trainer`) and adds one rule saying every member of it is a member of the broad class. PeTTaChainer refuses a narrowed class written *inside* another statement. For that case there is a stopgap: wmtm replaces it with one head-noun link per symbol. This costs no extra parse calls.
   - *Names.* A named individual mentioned in several notes is made into one entity, so "Mira" in note A and "Mira" in note B are the same symbol (Mesto §1).
   - *Placeholders.* When a note says "someone" or "a visitor", the parser invents a placeholder constant (a *skolem*, named `sk_...`). wmtm keeps the facts about placeholders but, by default, never asks the reasoner goals about them, because a conclusion about "some unnamed visitor" is rarely worth showing.
   - *Normaliser.* A clean-up pass turns certain flat rule forms into Inheritance links. It is in the code, but on the parser versions measured so far it had nothing to act on (see Status).
6. **Quality gate.** Before the atoms reach the reasoner's knowledge base, a gate removes facts that would make the reasoner say silly things (section 4.4 shows an example). It rejects facts:

- with parse confidence below 0.3;
- whose subject is a pronoun-like word or a placeholder;
- with a negative quantifier ("no falcon...");
- with a hedged quantifier ("mostly", "some", "often"), because chaining through "mostly" as if it meant "all" is unsound;
- that put something into a very broad class ("thing", "entity");
- that treat an identifier such as a commit hash as a class.

The gate also caps how many conclusions a single fact may support in one cycle (3).

7. **Cache.** The accepted atoms are written to the cache, keyed by the content hash. Each entry records the parser version and the prompt checksum, so stale parses can be found later.

There are three optional variants, all off by default:

- **Context parsing:** each note is parsed together with the atoms of up to three related earlier notes, so "she" in a later note can be tied to "Mira" in an earlier one. Every note is parsed a second time for this.
- **Speaker linking:** before parsing, "I", "me" and "the user" are rewritten to the resolved speaker's name.
- **Sentence mode:** the note is parsed one sentence at a time instead of whole.

The parser itself has two modes, *legacy* and the newer split mode. The live bot is believed to use legacy **[unverified]**.

4.4 Worked example

Note A, "Mira trains the falcons at the north tower.", goes into the queue under its content hash. The worker sends it to the parser. Suppose the parser returns:

```
(Member mira (Narrow trainer (Of falcon)))      conf 0.9
(Evaluation located_at mira north_tower)        conf 0.8
(Member sk_1 falcon)                            conf 0.7   ; "the falcons", an unnamed group
```

Lowering names the narrowed class `falcon_trainer` and adds the rule "every falcon_trainer is a trainer". The placeholder `sk_1` keeps its fact, but it will not be the subject of any goal. All three facts pass the gate. Note B, "Everyone who trains falcons at the tower keeps the lantern log", is parsed the same way. The useful atom from it is (Inheritance falcon_trainer lantern_keeper).

Now suppose Note B had said "falcon trainers *mostly* keep the lantern log". The parser would tag the link with the quantifier "mostly". The gate treats that as hedged and drops it by default, so the reasoner would never conclude that Mira certainly keeps the log.

A caveat about the example. In the parser runs measured so far, class-level statements like Note B's mostly did *not* come out as Inheritance links (see Limits). The example shows the path when they do.

4.5 Differences from the design doc, and why

The design (§9.3) asked for an "intentionally simple" subject-predicate-object extractor, and put full NLP out of scope. This is the largest deviation in the whole build. wmtm uses an LLM semantic parser because PeTTaChainer needs typed PLN atoms (Member versus Inheritance, quantifiers, rules) that a triple extractor cannot provide. The cost is real:

- every parse is an LLM call, so there is a budget (section 14);
- the same note parses differently from one run to the next;
- what the reasoner can do depends on what the parser chooses to emit.

The cache, the pinned prompt, the load check and the gate are all there to contain these costs.

4.6 Status

- The feed, the cache, the load check, the gate, the narrowed-class naming and its stopgap, name resolution and placeholder handling are all on main.
- Measured effect of the narrowed-class stopgap: on three frozen runs it recovered 76, 88 and 75 of the 99, 109 and 101 statements PeTTaChainer had refused. That took the candidate arm

from 0 proved goals to 12, 14 and 10.

- **Variance.** At a fixed parser version and prompt, all 20 evaluation notes parsed differently across three fresh runs. Atom totals were 810, 896 and 842, a spread of about 10 % of the mean. This is why evaluation uses frozen parses (section 15).
- **Modes.** In the split setup, legacy and split mode gave a byte-identical live context. Legacy kept more evidence for the reasoner (478 against 184 facts, 41 against 0 proved goals), at about 1.8 times lower token cost **[unverified: from the task record]**.
- **Sentence mode** was tried. It showed no clear coverage gain. It was more stable but less reliable, because the provider refused some calls. The conclusions it produced were all trivial echoes.
- A **master loader** for the newest parser version loads its output together with a seeded file of 280 rules. It exists on a task branch only, and the live path does not use it.

4.7 Limits

- **Class-level statements are mostly missing.** One run found 0 Inheritance atoms in the parser output. The parsers of that day emitted only Member facts. Without class-level links, chaining has little to chain. Whether the fix belongs in the parser (Man Hin) or in a wmtm-side fallback is undecided.
- **Run-to-run variance** of about 10 % means a single live run is not evidence of anything.
- **Questions are dropped.** The parser also emits queries ("who keeps the log?"). These are kept for the goal engine's need estimates, but no reasoner goal is ever formed from them.
- **Length.** Long notes are skipped rather than split, unless context or sentence mode is on.

Sources. [1] Feed, cache, preamble, prompt checksum, size limit and retries: `docs/RUNTIME.md` (Track H); `wmtm/parse_cache.py`; `wmtm/semparse_preamble.py`; flags `semparse_feed`, `semparse_preamble`, `parse_max_chars`, `parse_max_attempts`. [2] Gate: `wmtm/parse_gate.py` header (Tracks J, K, M, P); flags `gate_min_conf` (0.3), `gate_max_per_fact` (3). [3] Narrowed classes: `wmtm/pc_shadow.py:75-111` (design F7); stopgap `wmtm/semparse_adapter.py:473`, `:508`; t-2026-11-10-wmtm-narrow-preload (76/88/75 of 99/109/101; 0 -> 12/14/10). [4] Names and placeholders: `wmtm/name_resolve.py`, `wmtm/sk_registry.py`; flag `shadow_skolem_goals`. [5] Normaliser: `wmtm/parse_normalise.py`; t-2026-11-10-wmtm-normaliser-v2. [6] Context parsing and speaker linking: `docs/RUNTIME.md` (Tracks O, Q); flags `semparse_context`, `speaker_link`. [7] Modes: t-2026-11-10-wmtm-flag-semparse-legacy, t-2026-11-10-wmtm-gateway-callers, t-2026-11-10-wmtm-split-parser-modes-v2 (order attachment `wmtm-task-outcomes.md`). [8] Variance: t-2026-11-10-wmtm-parse-variance. Sentence mode: t-2026-11-11-wmtm-sentence-parse-v3, t-2026-11-12-wmtm-sentence-reasoning. [9] Class-level gap: t-2026-11-09-wmtm-flat-inheritance-v2. Master loader: t-2026-11-12-wmtm-parser-master-adopt-v2 (branch 785baf3, not on main). [10] Design: `docs/wmtm_design_doc.txt` §9.3.

5. The memory store and its items

5.1 Purpose

The store is the working memory itself: a bounded set of items, each carrying attention numbers, a usefulness score and a record of where it came from. Every other component either puts items into the store (recall, the reasoner, the request), reads it (the context block, the parse feed, the shadow worker), or takes items out (the forgetting policy). It sits at the centre of the data-flow diagram in section 1.

5.2 Inputs and outputs

- **In:** new items from recall, from reasoner admission, and the current request; "use" signals when the bot's reply relied on an item.
- **Out:** the active set sorted by attention (for the context block and the background workers); the list of items evicted on each tick (for the forgetting log); a saved snapshot of the whole store.

5.3 How it works

An item has:

- an id and its text;
- a *source type*: *recalled* (pulled from LTM), *derived* (a reasoner conclusion), *asserted* (the bot's own claim, treated like a note) or *user* (the current request);
- for a recalled item, the LTM cluster it came from; for a derived item, the ids of the items it was derived from;
- its three attention numbers (section 7), its age in ticks, its utility, and when it was last used;
- the wall-clock time it was first admitted, and the timestamp of the original journal entry if known;
- for a derived item, its *provenance*: which reasoner proved it, the goal, the truth value, the proof and the rules used (section 6);
- for the request item, metadata about who is speaking (used by speaker linking and the goal engine).

Admission. Putting an item in that is already there does not duplicate it. It boosts the existing item's STI instead and keeps the older origin timestamp. A new item starts with a given STI. If the store is full, the item with the lowest STI is evicted first, but the current request item is protected and goes last, so a burst of admissions in one turn can never push out the question being answered.

Ticks. Each cycle the store ages every item by one tick and decays its attention. Anything whose STI falls below a hard floor (0.01) drops out at once. The finer-grained forgetting rules (section 7) run after that.

Persistence. The bot's process does not stay alive between steps; it restarts every step. So the whole store is saved to a state file at the end of each turn and loaded at the start of the next. This is also why some rules use wall-clock time rather than tick counts: an item's tick age would otherwise never grow past one.

5.4 Worked example

At the start of the turn the store holds, say, 58 items. The request "Who keeps the lantern log this week?" enters as a *user* item. Recall admits Note B (keyword match) and Note A (graph link), both *recalled*, each with the STI recall gave it. The store now has 61 items and a capacity of 60, so the lowest-STI item that is not the request is evicted and handed to the forgetting log. On the next turn, if recall pulls Note B in again, Note B is not duplicated; its STI is simply boosted.

5.5 Differences from the design doc, and why

- The design (§3.1, §3.2) has the same shape: a bounded store of mutable items with attention values, source links and utility. The *user* and *asserted* source types are additions, needed so the request and the bot's own claims can be tracked separately from notes.
- The design's capacity is 200. The bot's default configuration uses 60. The reason is not recorded in the code; a smaller set keeps the context block and the shadow's knowledge base small **[unverified: reason]**.
- Overflow evicts by STI alone. The design (§7.1c) asked for a composite score with a 1.5x penalty for derived items. That was not built (it is one of the open gaps in the design's own §12 tests, section 16).
- Saving and restoring the store every turn is not in the design; it follows from how the bot runs.

5.6 Status

On main and always on (it is the core of wmtm). The design's §12 store tests pass: bounded size, decay, one eviction rule on every path, and derived items keeping their sources.

5.7 Limits

- Only STI decides overflow eviction; usefulness and being derived play no part there.

- The store does not resolve contradictions: two items that disagree can sit side by side (section 7).
- LTI is computed and saved but almost nothing reads it (section 8).

Sources. [1] Item fields and source types: `wmtm/item.py:14-46`. Store: `wmtm/store.py` (admit and re-admit boost `:43-87`; request item last `_evict_lowest_sti`; tick floor 0.01 in `tick`; snapshot `to_dict` / `from_dict`). [2] Default capacity 60: `wmtm/api.py:48`; design capacity 200: `docs/wmtm_design_doc.txt` §3.1. [3] Restart every step, wall-clock eviction: `wmtm/forgetting.py` (F15 note in `evaluate`). [4] §12 tests: `tests/test_design_s12_gapcheck.py` (19 passed + 5 xfailed at e59ab57); GAP-3 = composite eviction.

6. Truth values and sources

6.1 Purpose

A derived item is only as trustworthy as the evidence behind it. This component keeps two things with each conclusion: how strongly the reasoner believes it (the truth value) and what it rests on (its sources). Both are shown to the bot on the line that carries the conclusion, so the bot can weigh it, and both are used by admission and forgetting.

6.2 Inputs and outputs

- **In:** from the reasoner, a proved goal with a truth value (strength, confidence), a proof term and the ids of the items whose atoms the proof used.
- **Out:** a derived item whose provenance holds the truth value, the proof, the rules and the sources; a tag on its context line such as `derived tv=0.86,0.74 from=noteA,noteB`.

6.3 How it works

1. **Where the numbers come from.** Each parsed atom carries a truth value. PeTTaChainer combines premise truth values along the proof with PLN's formulas and returns one (strength, confidence) pair for the conclusion. wmtm never recomputes it; it stores what the engine gave.
2. **Strength as probability.** A strength below 0.5 means "probably not", not "weakly yes". By default such a result is skipped. An optional mode admits it with the text prefixed "unlikely:".
3. **No confidence cutoff.** A low-confidence answer is still an answer, so nothing is dropped for low confidence alone. Instead at most 5 conclusions are admitted per reasoner run, best first. Confidence is capped at 0.99.
4. **Sources.** The sources of a conclusion are the store items whose atoms the proof used. At admission at least one of them must still be in the store. A conclusion whose evidence has all been forgotten is not admitted.
5. **Display.** Each context line carries a tag: the item's kind, its truth value (derived items only) and up to four source ids. The tag deliberately leaves out attention, so the same item renders the same line whatever its current STI.
6. **Starting attention.** A conclusion's starting STI comes from how relevant its text is to the current request, not from its truth value: a base of 1.0 plus up to 4.0 more for full relevance. The base is high enough that a new conclusion is never evicted in the turn it arrives.

6.4 Worked example

The two premises arrive with their parse truth values, say (Member mira falcon_trainer) at (1.0, 0.9) and (Inheritance falcon_trainer lantern_keeper) at (0.95, 0.9). PeTTaChainer proves (Member mira lantern_keeper) and returns a combined value, for illustration (0.95, 0.81); the exact number depends on PeTTaChainer's formulas. The item is admitted as "mira is a lantern_keeper", sources {A, B}. Its context line reads like `mira is a lantern_keeper [derived tv=0.95,0.81 from=A,B]`. Its starting STI is high because "lantern_keeper" overlaps the request.

If the same proof had come out at strength 0.3, it would mean "Mira probably does not keep the log", and by default it would not be admitted at all.

6.5 Differences from the design doc, and why

- The design (§3.4, §5.3) had wmtm's own Python PLN compute truth values. That engine was removed in October; the truth values are now PeTTaChainer's own (section 12 explains what that does and does not give you).
- The design used confidence partly as an admission threshold. The build has no confidence cutoff, after Roman's point that the engine's low-confidence answer is still its answer; a per-run cap bounds the volume instead.
- The design (§12.2) pictured writeback as new EvidenceFor links in LTM. The code writes a different note form, and writeback is off by default (section 7).

6.6 Status

On main and active whenever the reasoner is. The truth value, proof and sources are recorded on every admitted conclusion and shown in the context block. Exact truth values are asserted in the tests where they matter.

6.7 Limits

- One pair per conclusion. Two independent proofs of the same conclusion are not merged into stronger evidence, because evidence aggregation is not present (section 12).
- Truth values are only as good as the parse confidences that feed them, and those come from an LLM.
- The source list names items, not the specific atoms within them.

Sources. [1] Engine truth value never recomputed: `wmtm/context.py:806-814` (`_engine_tv`); tag: `:826-838` (`_belief_tag`). [2] Strength 0.5 and negation mode: flags `reasoner_min_strength` (0.5), `reasoner_negation` (skip | unlikely); `wmtm/runtime_config.py:47-49`. Cap: `reasoner_max_admit` (5). No confidence cutoff: `wmtm/pc_admit.py:26-27`. [3] Live-source check, clamping, provenance: `wmtm/pc_admit.py` `load_candidates` (`:162-240`). [4] Starting STI: `wmtm/pc_admit.py:144-160` (`initial_sti`, base 1.0, span 4.0). [5] Design: §3.4, §5.3 (historical since Track L), §12.2.

7. Attention, admission, the trivial filter, and forgetting

This section covers everything that decides *how long* an item stays in the store: how attention rises and decays, what a derived conclusion has to pass to get in, which conclusions are refused as trivial, and how items are evicted and remembered as forgotten.

7.1 Purpose

The store is small on purpose. Something has to decide, every turn, what is worth keeping. The attention numbers measure recent relevance, and the utility score measures actual use. Admission keeps doubtful conclusions out. The trivial filter keeps out conclusions that say nothing new. Forgetting removes what has faded, and the forgetting log stops the reasoner from re-deriving the same thing in a loop. In the data-flow diagram, these rules wrap around the store box: admission on the way in, forgetting on the way out.

7.2 Inputs and outputs

- **In:** the store; the shadow worker's admit file; "use" signals from the bot's reply; the clock.
- **Out:** admitted derived items; refusals with a named reason (logged, counts only); evicted items; forgetting-log records; optionally, writeback notes to LTM (off by default).

7.3 How it works

Attention. Each item carries three numbers, after OpenCog's ECAN:

Number	Meaning	Kept per tick
STI	relevance right now	90 %
ATI	medium-term importance	97 %

Number	Meaning	Kept per tick
LTI	long-term importance	99.5 %

When an item is used or re-admitted it gets a *boost*. Most of the boost goes into STI, 5 % of it goes into ATI, and 2 % of the existing ATI moves into LTI. So an item that keeps being used slowly builds up importance that outlasts any single conversation. For ranking, the three are combined as STI + 0.5 × ATI + 0.2 × LTI.

Utility. Utility counts real use: 2 points each time the item is used, 1.5 points each time a conclusion was derived from it, minus 0.3 for each turn it sat in the context without being used. It never goes below zero.

Admission of a derived conclusion. The shadow worker (section 9) has already thrown away anything that failed its own checks, and writes only the survivors to the admit file. On the next turn, WMTM reads that file and admits from it only if *all* of these hold:

1. the file exists and parses;
2. it is fresh: written no more than 15 minutes ago;
3. its run finished normally, the canary was proved and the negative control was not;
4. this run has not already been admitted from (each run is consumed once);
5. per conclusion: the proof is a real derivation, not just an asserted fact read back;
6. per conclusion: it is not trivial (below);
7. per conclusion: its strength is at least 0.5 (section 6);
8. per conclusion: at least one of its sources is still in the store;
9. per conclusion: it is not already in the store and is not a derivation that was forgotten.

At most 5 conclusions get in per run, best first. Every attempt, admitted or not, is logged with counts and reasons and no text.

The trivial filter. Many proofs are technically valid but useless. The filter looks only at the *shape of the proof*, never at the words, and refuses a conclusion when every step of its proof is one of two kinds, applied directly to asserted facts:

- **head-noun step:** "every falcon_trainer is a trainer", applied to "Mira is a falcon_trainer", gives "Mira is a trainer". This only drops a modifier the note already contained.
- **narrowing echo:** a rule whose premise class already contains its conclusion, such as "anything that is a staged commit is staged".

Any other rule, any second step through a derived premise, or anything the filter does not recognise means *not trivial*, and the conclusion is admitted. The filter fails open on purpose: it may let through a dull conclusion, but it should never block a real one.

Forgetting. Several rules evict items. Since November they all run on every path (full cycle, fast path, cleanup) and record their evictions in the same log:

- STI below a floor: 0.05 for normal items, 0.10 for derived ones.
- Old and unused: after 50 ticks (20 for derived items), an item with utility below 0.5 goes.
- Wall-clock staleness: because the bot restarts every step, tick ages stay small, so stale-cleanup also uses real time. A derived item with no utility goes after an hour, and one with little utility after a day. A recalled note gets much longer (no utility: 48 hours; little utility: 7 days; hard cap 14 days), so that a note does not cycle forever between being evicted and recalled again. A note that was recalled in the current turn is protected from cleanup in that turn.
- Capacity: over capacity, the lowest-STI item goes, with the current request last.

The forgetting log. Each eviction is recorded with a *derivation key*: a hash of the conclusion's text together with its sorted source ids. Admission checks this key. The same conclusion from the same sources is never admitted again, but the same conclusion reached from *different* sources is a new derivation and can come back. The log keeps the newest 500 records.

Writeback. A recalled item that survives 30 ticks with utility 2.0 or more, or a derived item that survives 15 ticks with utility 1.5 or more, can be written back to the LTM journal as a note. For derived items this is switched off by default, so unreviewed conclusions do not become

permanent.

7.4 Worked example

The conclusion "mira is a lantern_keeper" arrives in the admit file from a run 4 minutes old, whose canary passed. Its proof uses the rule from Note B, a real class-level rule, so it is not trivial. Its strength is 0.95, and both sources are still in the store. It is admitted with starting STI 1.0 plus a relevance bonus, say 3.4 in total.

In the same file is "mira is a trainer". Its only step is the head-noun rule applied to the asserted fact from Note A, so the filter refuses it as `trivial_restatement`, and the refusal is logged.

The conversation moves on. Each tick keeps 90 % of the STI: 3.4, 3.06, 2.75, ... Left alone it would fall below the 0.10 derived floor after about 34 ticks, but the 20-tick age rule for little-used derived items, or the one-hour wall-clock rule, usually takes it sooner. It is evicted and its derivation key (text + {A, B}) is logged. The next reasoner run proves the same thing again from A and B, and admission drops it as *known*. A week later a new Note C says Mira was put on the night-watch rota, and Note D says everyone on the night watch keeps the lantern log. The conclusion is proved again from {C, D}. That is a different key, so it is admitted.

7.5 Differences from the design doc, and why

- **Decay rates.** The design (§10) says STI decays by 2 % a cycle and spreading activation decays by 0.7. The build keeps 90 % per tick and uses 0.3 for spreading. These were tuned in earlier tracks and never re-derived from the design.
- **Eviction score.** The design (§7.1c) ranks by a composite of STI, utility and age, with a 1.5x penalty for derived items. The build uses STI for overflow plus the separate thresholds above. This is simpler, and it has not yet been redone (open gap).
- **Forgotten conclusions in LTM.** The design (§6.3) writes a "derived-forgotten" note to LTM. The build keeps the forgetting log in the store's saved state only (open gap).
- **Writeback.** The design (§6.3, §13.3) promotes durable derived beliefs to LTM. The build has the mechanism but keeps it off, until there is a signal that tells useful conclusions from trivial ones.
- **Trivial filter, freshness and canary gates.** These are not in the design. They were added after measurements showed that most admitted conclusions were restatements, and that a broken reasoner run could otherwise leak results.
- **Wall-clock rules.** These are not in the design. They follow from the bot restarting every step.

7.6 Status

- Everything above is on main.
- The trivial filter is on by default. Admission from the reasoner is on whenever the reasoner is (it is the default reasoner). Writeback of derived beliefs is off.
- One forgetting rule on every path is on by default.
- The design's §12 forgetting tests pass. The tests for composite eviction, writeback form and the forgotten-note are expected failures that name their gaps.

7.7 Limits

- The trivial filter catches only the two proof shapes above. Restatements that go through a parser-made rule still get through. In the latest parser run, 82 of 105 proved goals were restatements of one kind or another (section 15).
- Utility credits an item when it is *shown* in the context, not when the reply provably used it. So "useful" is only approximated (the open part of the design's useful-vs-trivial gap).
- No contradiction handling: if one note says the log moved to the gatehouse and another says it did not, both stay.
- The forgetting log is capped at 500 records. A conclusion forgotten long enough ago can be re-admitted from the same sources.

Sources. [1] Attention: `wmtm/attention.py` (decays 0.90/0.97/0.995, flows 5 %/2 %, total weights 1/0.5/0.2). Utility: `wmtm/utility.py:26-34`. [2] Admission: `wmtm/pc_admit.py` header and `load_candidates`; flags `reasoner_max_age_s` (900), `reasoner_max_admit` (5); `docs/RUNTIME.md` "Reasoner switch". [3] Trivial filter: `wmtm/pc_admit.py:322-337`, `trivial_reason :455`; orchestrator guard `wmtm/orchestrator.py:262-275`; flag `admit_trivial_filter` (default on); t-2026-11-10-wmtm-trivial-filter (commit 8849379), t-2026-11-10-wmtm-trivial-filter-followup-v2. [4] Forgetting thresholds: `wmtm/forgetting.py` (0.05/0.10, 50/20, 0.5, 6 h zero-utility); stale cleanup `wmtm/context.py:110-225` (recalled tiers 48 h / 7 days / 14 days); recall protection `cleanup_protect_recent` (`docs/RUNTIME.md` "Context block"); one rule everywhere: flag `forget_one_rule`, `wmtm/api.py:320`, `:345`. [5] Forgetting log and derivation key: `wmtm/forgetting_log.py:30-40`, cap 500. [6] Writeback: `wmtm/writeback.py:73-120` (30 / 2.0; 15 / 1.5); flag `reasoner_writeback` (false). [7] Design: §6.3, §7.1c, §10, §13.3; gaps GAP-3, GAP-4, GAP-7, GAP-8 (design-gap-check). [8] 82 of 105 restatements: t-2026-11-12-wmtm-parser-master-adopt-v2.

8. Recall and the context block

8.1 Purpose

Recall decides which notes from the large LTM journal come into the store this turn. The context block decides which resident items the bot actually sees. Together they are the part of WMTM that the bot feels directly: if recall misses a note, the bot does not know it; if the block buries an item, the bot does not use it. In the data-flow diagram, recall is the arrow from the LTM journal into the store, and the context block is the arrow from the store to Omega.

8.2 Inputs and outputs

- **In:** the current request text; the LTM journal, read as clusters of notes with their links (evidence-for, promotes-from, supersedes); the items already in the store.
- **Out:** up to 10 newly recalled items per cycle, each with a starting STI; the context block, a short ranked list of item lines, each with its tag (section 6).

8.3 How it works

Keyword recall. Each LTM cluster is scored against the words of the request in layers:

1. overlap of words, with rare words counting more than common ones;
2. a small bonus for each longer request word found inside the cluster text;
3. a bonus for matching consecutive word pairs;
4. a bonus for clusters that other clusters cite as evidence;
5. spreading activation from items already in the store (below);
6. a recency factor: notes from the last day get 30 % more, from the last week 10 % more, and notes older than two weeks lose 20 % per extra week, down to 30 % of their score.

Superseded clusters are never candidates; they are removed when the journal is read.

Graph recall. Notes are linked in LTM. A note can be *evidence for* another, or be *promoted from* another. Spreading activation starts at the items already in the store and follows these links. A direct link to a resident item is worth 0.5. Activation then spreads along evidence-for links to depth 2, losing most of its strength at each hop, and along promoted-from links to depth 1. Until November, this spread could only *reorder* clusters that already had a keyword hit. Now a cluster with no shared word can be recalled on its links alone, as long as its score clears a floor of 0.1.

Budget. At most 10 new items are recalled per cycle, over all recall paths together.

Where the spread starts. A setting chooses the seeds: every resident item (the default), the top half by STI, or only this turn's keyword hits plus the request. See Status for why this matters.

The context block. On a real user turn, every resident item gets a relevance score against the request (the same scoring as recall). The block ranks items by a weighted sum of relevance and STI (STI scaled by the highest STI in the store), and gives each item a longer, request-focused excerpt of its text. Freshly recalled notes are protected from stale-cleanup in the turn they were

recalled, so they cannot be evicted before the bot sees them. After the item lines come, optionally, at most three goal-suggestion lines (section 10).

8.4 Worked example

The request is "Who keeps the lantern log this week?". Keyword recall scores Note B high: "lantern" is rare in the journal, "log" less so, and "lantern log" also matches as a pair. Note A, "Mira trains the falcons at the north tower", shares no word with the request, so keyword recall alone scores it 0. Note B has just become resident, and in LTM Note A is recorded as evidence for Note B. That direct link gives Note A 0.5, which is over the floor, so graph recall admits it too. Without graph recall the bot would see only Note B ("everyone who trains falcons at the tower keeps the log") and could not name anyone.

In the context block both notes rank near the top: B for its relevance, A because it is new and has high STI.

The same mechanism can also go wrong. If the store holds forty old items, all of them are seeds by default, and their links can pull in many notes that have nothing to do with lanterns. Those notes then push useful items out.

8.5 Differences from the design doc, and why

- **Spreading decay.** The design (§10) uses 0.7 per hop. The build uses 0.3. This was tuned early and not revisited.
- **Graph recall.** The design's "key insight" (§3.3) is that links should bring in notes with no shared words. The first build only reordered keyword hits. That was fixed in November (gap GAP-1). Promoted-from links were read but not followed until the same fix (GAP-2).
- **Budget and floor.** These match the design's §8.2 and §8.3. Both were added late. Before, the per-turn limit was 15.
- **The context block** is not specified in detail by the design. Ranking by relevance plus STI, with longer excerpts, was added in the rc3 release because ranking by attention alone showed old, high-STI items over the ones the request was about.
- **Recency factor and the same-turn protection** are not in the design. Protection was added after a release in which freshly recalled old notes were cleaned up in the same turn, before the block was built.

8.6 Status

- Keyword recall, graph recall, the floor, the budget and the ranked context block are all on main, and on by default.
- **Graph-only recall is not ready as a default.** A replay found it lowered the share of useful admitted conclusions from 56 % to 14 %, with 471 admissions against 32, and it churned the store. The budget of 10 cut admissions (339 against 471) and evictions (243 against 383), but not that share.
- **Seeding only from this turn's hits** removed the churn (0 evictions against 243) and raised the share of useful conclusions (0.52 against 0.15). But then nothing was recalled purely through links on turns without keyword hits, which was the point of graph recall. Which seeding to use by default is an open decision for andy.

8.7 Limits

- Keyword matching is literal. "Falconer" does not match "falcon trainer".
- Recall uses the journal's own links. Notes with no links can only come in by keyword.
- Relevance is lexical, so the context block can rank a note that shares many words above one that answers the question.
- The trade-off between graph reach and churn is unresolved (Status).

Sources. [1] Scoring layers and recency: `wmtm/recall_bridge.py` `_score_cluster` (`:292-380`); superseded dropped `:77-86`, `:127-160`. [2] Graph recall: `recall_bridge.py` `recall` (`:206-255`), `_spreading_activation_for` (`:542-570`, direct weight 0.5), `spreading_activation` (`:463-`);

t-2026-11-10-wmtm-graph-recall, -graph-recall-floor-replay. [3] Floor and budget: flags `recall_relevance_floor` (0.1), `recall_budget` (10), `runtime_config.py:169-170`; `wmtm/context.py:716-727`; t-2026-11-11-wmtm-recall-budget-10. [4] Seeds: flag `recall_spread_seeds` (all | focus | turn), `context.py:730-741`; t-2026-11-11-wmtm-spread-seeds-probe. [5] Context block: `docs/RUNTIME.md` "Context block (rc3, Track P)"; flag `context_rank` (true); `cleanup_protect_recent`. [6] Graph-only replay: t-2026-11-11-wmtm-graph-only-probe. [7] Design: §3.3, §8.1-8.3, §10, §13.5.

9. The reasoner: PeTTaChainer in the shadow worker

9.1 Purpose

The reasoner joins facts that sit in different notes. Recall can put Note A and Note B side by side, but only a reasoner can say what follows from the two together. WMTM uses PeTTaChainer, a backward chainer for PLN written in MeTTa. It runs in a separate background process called the *shadow worker*, so a slow or stuck proof never holds up a turn. In the data-flow diagram it is the "shadow worker" box: atoms come in from the parse cache, and the admit file goes out to admission (section 7).

9.2 Inputs and outputs

- **In:** a *spec* written by the turn: the top items of the store (up to 200 by attention) with their ids, STI and source type, plus the current request item.
- **Out:** a run record (status, canary result, every goal asked, time per goal, how many were proved); and the admit file, holding only the proved conclusions that passed every check of the run, each with its truth value, proof and source items.

9.3 How it works

1. **Launch.** At the end of a cycle, WMTM copies the top items into a spec and starts the worker as a detached, lowest-priority process. It never waits for it. At most one worker runs at a time. If one is still running, this cycle simply does not launch another. A worker older than 15 minutes is presumed hung and is killed by the next launch.
2. **Build the knowledge base.** The worker looks up each item's atoms in the parse cache (section 4). Items that are not parsed yet contribute nothing this time. It adds the lowered facts, the parser's rules, the defining rules for narrowed classes, and a tiny fixed *canary* problem about a made-up person called Grace. The quality gate (section 4.3) is applied here. wmtm adds no inference rules of its own: all reasoning is PeTTaChainer's.
3. **Find candidate goals.** A goal here is a statement to prove (the reasoner sense). The worker looks for statements that *could* follow by chaining: Member-then-Inheritance steps and Inheritance chains, up to 3 steps long, that are not already asserted. By default goals about placeholders (skolems) are left out. This gives a pool of up to 5000 candidates.
4. **Rank the goals.** Each candidate gets a score: 2 × (overlap with the current request) + 1 × (chain length - 1) + 1 × (how informative the class is) + 0.5 × (STI of its source items). A class that just echoes the subject's own name counts as uninformative. The best 60 goals are asked.
5. **Canary and negative control.** Before trusting any answer, the worker asks two fixed questions through the same process: "is Grace a person?" must be proved, and "is Grace a robot?" must *not* be. If either answer is wrong, the run is marked as failed and none of its results is trusted. This is the same check the experiment sanity rules ask of every batch run.
6. **Ask.** Each goal is sent to the PeTTaChainer process with a hard time limit (2 seconds by default). A goal that times out kills the process, which is restarted for the rest, within an overall budget. Each answer is checked for structure (a proof term and a truth value that parse).
7. **Write.** The run record always gets written, including failed runs. The admit file gets only conclusions from a run whose canary passed, that were proved in time and are structurally sound. Admission (section 7) applies its own checks again on the next turn.

PeTTaChainer and PeTTa are pinned to fixed versions, so a change upstream cannot silently change results.

9.4 Worked example

The spec holds, among others, Note A, Note B and the request "Who keeps the lantern log this week?". The knowledge base gets (Member mira falcon_trainer), (Inheritance falcon_trainer lantern_keeper), the rule "every falcon_trainer is a trainer", the placeholder fact about the falcons, and the Grace canary.

The goal pool includes (Member mira lantern_keeper), 1 step; (Member mira trainer), 1 step, through the defining rule; and nothing about sk_1. Ranking puts (Member mira lantern_keeper) first: "lantern" and "keeper" overlap the request, and "lantern_keeper" is informative. (Member mira trainer) scores lower, because "trainer" is already part of "falcon_trainer".

The canary is proved, Grace is not a robot, and both goals are proved within the time limit. Both go into the admit file. On the next turn admission takes the first and the trivial filter refuses the second (section 7.4).

If, instead, the PeTTaChainer process had come up broken and proved "Grace is a robot", the run would be marked `canary_fail`, the admit file would be empty, and the turn would admit nothing.

9.5 Differences from the design doc, and why

- **Which engine.** The design (§3.4, §5) put a small PLN engine *inside* WMTM, written in Python, running deduction, induction, abduction and revision in each cycle. The build first did that, then ran PeTTaChainer next to it in "shadow" mode for comparison, and in October removed the Python engine. PeTTaChainer is the project's real PLN reasoner, and keeping a second, simpler one meant two sets of truth-value formulas that disagreed.
- **Outside the turn.** The design ran inference inside the cycle with a step budget. PeTTaChainer runs on PeTTa, which starts slowly and can take seconds per goal, so it runs in the background and the turn reads its last result. The cost is that a conclusion arrives one turn late.
- **Goal-directed, not forward.** The design's engine applied rules forward to everything. A backward chainer needs goals, so the build added goal finding and ranking. Ranking by request overlap is what makes the conclusions relevant to the conversation.
- **Canary.** Not in the design. Added under the project's experiment sanity rules after an engine path failed silently.

9.6 Status

- On main. PeTTaChainer is the default reasoner; it can be switched off by a flag without a restart. There is no fallback engine.
- Ranked goals, the canary, per-goal time limits and the admit file are all on.
- The worker's knowledge base is built only from cached parses, so the reasoner is only as active as the parse feed (sections 4 and 14).
- How much it contributes is measured in section 15. In short: it does prove goals and admit conclusions, but most of what it proves restates the notes, and the useful share depends heavily on the parser.

9.7 Limits

- **One turn late.** A conclusion relevant to this turn's question appears, at best, on the next turn.
- **Only short chains.** Up to 3 steps. Transitive chaining runs only along Inheritance links. The parser's Implication rules are applied to Member facts one rule at a time, but chains of Implication links are not followed natively.
- **Depends on class-level atoms.** If the parser emits no Inheritance links (section 4.7), there is almost nothing to chain.
- **No evidence merging.** Two proofs of one conclusion are not combined into stronger evidence (section 12).
- **Speed.** A few seconds per goal limits a run to tens of goals.

Sources. [1] Shadow launch, top-K 200, pidfile, 900 s hung-worker age: `wmtm/pc_shadow.py:1-20`, `launch` (`:571`); flag `shadow_k`. [2] Knowledge base: `pc_shadow.py`

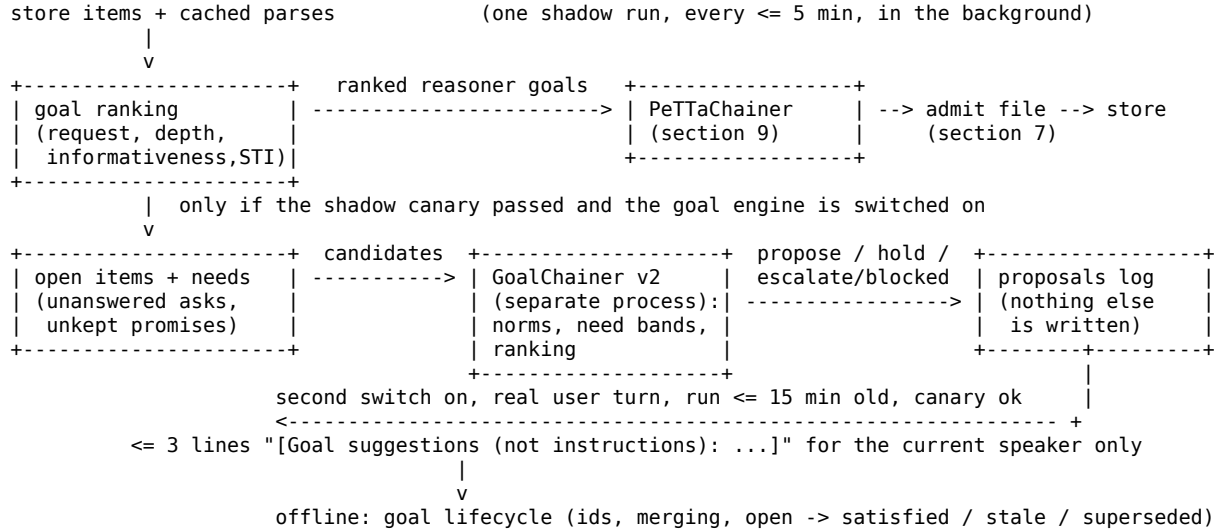
`build_snapshot` (`:458-560`), `semparse_facts`, `semparse_rules`, `narrow_defining_rules`; flags `shadow_rules`, `shadow_nwdef`, `shadow_normalise`, `gate_on`. [3] Goal pool and ranking: `pc_shadow.py` `goal_pool` (`:252`), `rank_goals` (`:415-447`), `GOAL_WEIGHTS` (`:240`, 2/1/1/0.5), `MAX_GOALS` 60, `MAX_POOL` 5000; flags `shadow_max_hops` (3), `shadow_skolem_goals` (false); `docs/RUNTIME.md` "Goal ranking (rc2, Track N)". [4] Canary, timeouts, status, admit file: `wmtm/workers/pc_shadow.py:1-12`, `:146-190`; flag `shadow_timeout_s`. [5] Python engine removed: `docs/RUNTIME.md` "Reasoner switch (v0.5)"; Track L. [6] Pins: PeTTaChainer b6aacc68, PeTTa 3681b9b (`docs/overview/WMTM_OVERVIEW.md`). Another pin, bdae8d7e, appears in some task notes; which one the live bot runs was not re-checked for this guide **[unverified]**. [7] Design: §3.4, §4, §5.

10. Goals: the goal ranking, the goal engine, and the goal lifecycle

The word *goal* means two different things in wmtm, and this section covers both.

- A **reasoner goal** is a statement the shadow worker asks PeTTaChainer to prove (section 9). Choosing *which* statements to ask is how the current conversation steers reasoning. This is "goal-driven guidance" in the reasoning sense.
- An **agent goal** is something the bot could *do*: ask a clarifying question, remind someone of a promise, flag an uncertainty. The **goal engine** (built on MesTTo's GoalChainer v2) proposes these. The **goal lifecycle** tracks them over time.

The goal loop, as built:



10.1 Purpose

The goal ranking exists because the shadow worker can only ask about 60 questions per run, and a knowledge base of a few hundred facts yields many more possible chains. Without ranking, the questions would be about whatever happens to come first; with it, they lean toward what the person is asking about, toward conclusions that need real chaining, and away from echoes.

The goal engine exists because a working memory can notice things a single reply does not: a question someone asked three days ago that nobody answered, a promise the bot made and has not kept. The engine turns those into proposed actions, checks each against rules about what the bot may do, and ranks them. It is step 4 of your plan of 12 October ("PeTTaChainer as the reasoner, the semantic parser as the parser, GoalChainer on the same space, then goals").

The lifecycle exists so that the same goal, proposed many times in different words, is recognised as one goal, and so that a goal can be closed when later notes show it was done.

10.2 Inputs and outputs

- **Goal ranking.** In: candidate chains from the knowledge base, the current request item, and item STI. Out: an ordered list of at most 60 reasoner goals, each with its score parts recorded in the run log.
- **Goal engine.** In: the snapshot's items and the parser's cached *questions*. Out: records in a proposals log (action type, the need it serves, the wording, why, score, legitimacy, status, the item ids it rests on, who it is for), a run log and a "latest run" file. Optionally, at most 3 suggestion lines in the bot's context.
- **Lifecycle.** In: copies of the proposals log and the journal. Out: a goal table with ids and states. It runs offline only; the live bot never calls it.

10.3 How it works

Goal ranking (reasoner goals). Each candidate chain gets

> score = 2 × request overlap + 1 × (steps − 1) + 1 × informativeness + 0.5 × source STI

- *Request overlap* is 1 if the subject or the class is an entity the current request is about (an entity parsed from the request, or one link away from one). Otherwise it is the share of the goal's name tokens that occur in the request text. So an unparsed request still steers by name.
- *Steps* is how many derivations the chain needs (1 to 3). Longer chains are rewarded because one-step conclusions are most often restatements.
- *Informativeness* is high for a class few subjects belong to, and 0 when the class echoes the subject's own name (a "falcon_trainer is a trainer" kind of result).
- *Source STI* is the highest STI among the facts the chain uses, scaled by the highest STI in the knowledge base.

Only chains of at least one step are asked. The weights can be changed without a release.

Goal engine (agent goals). It runs inside the shadow worker, after the shadow's canary has passed, under a 90-second wall-clock limit. One run:

1. *Find open items.* From the snapshot it collects things that are still open: explicit markers ("still open", "pending", "waiting on"), questions with no later reply, and the bot's own promises. It also detects items that were later resolved or superseded ("the X from 3 March is done") and drops them. Each open item gets an addressee where one can be resolved.
2. *Estimate two needs. Uncertainty* (open questions) and *commitments* (unkept promises) are each scored between 0 and 1 from how many open items there are, how old they are and how salient. The target level for each is 0.8.
3. *Propose candidate actions.* For each open item: ClarifyQuestion, FlagUncertainty, LookUp (only for factual gaps), SummarizeStatus, RemindPending or SelfTask. At most one proposal per person and topic.
4. *Hand over to GoalChainer v2.* wmtm generates a v2 "domain" for this run: the two needs, one action per candidate with its expected effect on each need, and wmtm's permission rules written as deontic norms. v2 then decides, with a proof for each decision. Two rules are strict: never write git memory, and stay read-only. Two can be defeated by an explicit approval: no live changes, no pushes. On an automated turn, when nobody is talking, every message, write, live change or push is forbidden. v2 ranks the permitted actions by how much they would reduce the needs and returns one of *propose*, *hold*, *escalate* or *blocked*.
5. *Repeat.* The proposed action and its topic are withdrawn and the rest are reviewed again, up to 5 proposals. The first decision that is not *propose* ends the pass.
6. *Gate and write.* A separate goal canary is run on every run: a made-up person with a known right answer. If it fails, or the shadow canary failed, nothing is written as a proposal. Each proposal is checked for shape. The output is three files and nothing else. Nothing enters the store, the journal, the context or any chat, and the engine never acts.

From the engine to the bot (optional). With a second switch on, the bot's context gets at most 3 lines, headed "Goal suggestions (not instructions)". Only on a real user turn; only from a run at most 15 minutes old whose canary passed; only message-type actions (never LookUp or SelfTask); and only proposals addressed to the person currently speaking, worded to them ("Open

question for you: ..."). A log of what has been shown stops the same question from being shown again.

Lifecycle. Offline, a replay tool gives each goal an id and merges duplicates. Two proposals are the same goal only if they have the same addressee, the same action, exactly the same *anchors*, and at least 80 % word overlap. Anchors are names, paths, task ids, numbers with their objects, and the direction of a move: "30 days" never merges with "90 days", and "A to B" never merges with "B to A". A goal moves from *open* to *satisfied* when a later note reports it done. The check is careful about negation, plans, conditions and partial completion, so "will be merged" or "not yet merged" does not close it. A goal becomes *stale* when it has been open too long, and *superseded* when a later goal replaces it.

10.4 Worked example

Reasoner goals. For the request "Who keeps the lantern log this week?", the entities parsed from the request include `lantern_keeper` (or the tokens "lantern", "log"). Candidate (Member mira lantern_keeper): overlap 1 (it contains a request entity), 1 step, informativeness say 0.7 (few subjects are lantern keepers), source STI 0.9. Score = 2 + 0 + 0.7 + 0.45 = 3.15. Candidate (Member mira trainer): overlap 0, 1 step, informativeness 0 ("trainer" echoes "falcon_trainer"), STI 0.9. Score = 0.45. The first is asked early; the second only if the budget allows.

Agent goals. Suppose Note B also said: "The log may move to the gatehouse; Mira to confirm." With nothing later from Mira, this is an open question addressed to Mira, and the uncertainty need rises above its target. The engine proposes ClarifyQuestion for Mira about the log's location. v2 permits it on a human turn, since a message to the person asking is allowed, and proposes it. The proposal goes to the log. If the suggestion switch is on and Mira herself writes to the bot within 15 minutes, her context gets one line: "[Goal suggestions (not instructions): Open question for you: does the lantern log move to the gatehouse?]". If someone else is speaking, nothing is shown. On an automated turn the same proposal would be forbidden.

Lifecycle. A week later a note says "Mira confirmed: the log moved to the gatehouse." The replay closes the goal as satisfied. Had the note said "Mira will confirm the move", it would stay open: a plan is not a completion.

10.5 Differences from the design doc, and why

- The design doc has no goal engine. This was added for step 4 of the 12 October plan.
- The design's §11 uses "goals" only to boost attention for items matching an active task. A boost of that kind is in the code: items whose words match the active goal's description get extra STI each cycle. It relies on an active goal being set, and I could not confirm that anything sets one on the live path **[unverified]**.
- The first GoalChainer integration (an incident-response demo with fixed actions) was replaced by v2 run as a separate process, because the demo ignored wmtm's evidence and its own PeTTaChainer call overflowed the stack. GoalChainer is kept out of wmtm's own process for licensing reasons as well (it is GPL-3.0).
- The goal ranking is the first of three designs proposed under "Track M". The other two are not built: goals made from the parser's open questions, and a one-line contradiction check.

10.6 Status

- Goal ranking: on main, on by default whenever the shadow runs.
- Goal engine: on main, **off by default**, log only. The suggestion lines are a second switch, also off.
- Lifecycle: on main, offline tool only. It is tested against an adversarial set of 31 merge pairs and 27 closure cases.
- Measured: **no A/B test has exercised the goal engine.** The last A/B and the latest parser run both had it switched off. Its tests (it reads wmtm's evidence, ties stay ties) pass; the test against the real v2 runtime is skipped where that runtime is not installed.

10.7 Limits

- The engine finds open items from text markers and reply patterns, not from understanding. It misses open items that are phrased unusually, and it can treat a rhetorical question as open.
- It proposes, it never acts, and its only route to the bot is three suggestion lines that are off by default. So today it guides nothing on the live bot, unless that switch is on there **[unverified: live flags]**.
- Its value is unmeasured.
- The goal ranking steers *which* conclusions are sought, but if the parser gave no class-level links, the best-ranked goals are still restatements (section 15).

Sources. [1] Goal ranking: `wmtm/pc_shadow.py:415-447` (score = w_req·overlap + w_depth·(hops−1) + w_info·info + w_sti·sti); flag `shadow_goal_weights` "2,1,1,0.5" (`wmtm/runtime_config.py:158`); `docs/RUNTIME.md` "Goal ranking"; Track N. `docs/TRACK_M_GOALS.md` (Designs A, B, contradiction check). [2] Goal engine: `docs/RUNTIME.md` "Goal engine"; `docs/GOALS_V2.md` (layers, per-run steps 1-8, norms `no_git_memory`, `read_only`, `no_live_change`, `no_push`, `no_autonomous_<effect>`); flags `goalchainer` (false), `goal_engine` (v2), `goal_weights` (1,1), `goal_targets` (0.8,0.8), `goal_max_proposals` (5), `goal_budget_s` (90) (`wmtm/runtime_config.py:68-83`); worker hook `wmtm/workers/pc_shadow.py:193`. [3] Suggestions: flags `goal_prompt` (false), `goal_prompt_max_age_s` (900), `goal_prompt_max_lines` (3) (`wmtm/runtime_config.py:91-93`); `wmtm/goal_prompt.py`; Tracks S, T. [4] Lifecycle: `docs/GOAL_LIFECYCLE.md`; `wmtm/goals/lifecycle.py`; `tests/test_goal_lifecycle.py`. [5] History: `docs/GOALCHAINER_DESIGN.md` (16 Oct proposal: demo limits, stack overflow, evidence ignored). [6] Active-goal boost: `wmtm/orchestrator.py:235-246` (`_goal_context_boost`). Old bridge phase `:390-414` (`use_goalchainer`). [7] Not measured: t-2026-11-08-wmtm-f3-ab-v3 ("criterion 5 was not exercised"), t-2026-11-12-wmtm-parser-master-adopt-v2.

11. Induction

11.1 Purpose

Deduction (section 9) goes from general to particular: every falcon trainer keeps the log, Mira is a falcon trainer, so Mira keeps the log. Induction goes the other way. If many individuals turn out to be in both class A and class B, it guesses the general link "every A is a B", with a confidence that grows with the number of examples. The design (§9.1b) asks for this, so that the working memory can notice regularities that no single note states. In the flow, induction is an extra source of reasoner goals inside the shadow worker. Its answers then go through the same admission rules as any other conclusion.

11.2 Inputs and outputs

- **In:** the Member facts in the shadow's knowledge base.
- **Out:** extra reasoner goals of the form (Inheritance A B), added to the goal pool. When PeTTaChainer proves one, the result is an ordinary candidate conclusion, with a truth value and with the shared instances as its sources.

11.3 How it works

1. For every pair of classes A and B, the worker counts the individuals that are members of both.
2. A pair becomes a goal only if it shares at least 3 individuals (the threshold can be changed), the link is not already asserted, neither class is an invented placeholder, and the two classes are not just a narrowed class and its own base (that link is true by definition and says nothing).
3. Only one direction is asked per pair: from the class whose members are more fully covered by the shared set.
4. PeTTaChainer has no separate inheritance-induction rule at the pinned versions. It answers such a goal with a step called the *instance fold*. That step averages the evidence of the individual instances, each weighted by its own strength and confidence, exactly as §9.1b describes. Confidence grows only slowly with each extra instance: about 0.001 per instance at the pins.
5. The goal gets the induction source (one step) and is ranked with the others (section 10.3).

11.4 Worked example

Suppose the store held notes saying Mira, Tomas and Ilse are each falcon trainers and each on the night watch, and nobody had written "falcon trainers are on the night watch". The classes `falcon_trainer` and `night_watch` share three members, which meets the threshold, so the worker asks (Inheritance falcon_trainer night_watch). PeTTaChainer folds the three instances and proves it with strength near 1.0 but very low confidence, a few thousandths. That is honest: three examples are weak evidence for a general rule. If admitted, the conclusion would carry that low confidence on its context line, and the bot could weigh it accordingly.

With only Mira and Tomas in both classes, there would be no goal at the default threshold.

11.5 Differences from the design doc, and why

The design's description (a weighted average over instances) is what the engine computes, so the mechanism matches. The difference is in reach. The design expects induction to be a routine source of new beliefs. In the build it is a switch, off by default, because on the measured data it produced nothing useful (below).

The design also implies a second path: two rules that share structure combined into a conditional link. The engine has that path, but it needs base rates (how common each class is), and wmtm does not supply any. Without them the path gives no answer, or a confidence around 0.016.

11.6 Status

- On main, **off by default**.
- Measured on the frozen parses: at the default threshold of 3 shared instances, induction added no conclusion at all. At a threshold of 2 it added one rule on three of the parses, built from two notes' own placeholder individuals, at confidence 0.0024. Every such rule was judged UNCLEAR, because the store can neither confirm nor refute it. The candidate arm failed the count rule against the baseline.

11.7 Limits

- The parser emits few clean Member facts about *named* individuals. Many instances are placeholders, which the goal builder skips, so pairs rarely reach the threshold.
- Confidence from a handful of instances is tiny, so even a correct induced rule is barely usable downstream.
- No base rates, so the conditional-link path is unused.
- Induction can easily produce coincidences ("everyone who did the cleanup is done"). Nothing beyond the threshold and the low confidence guards against that.

Sources. [1] Goal pool: `wmtm/pc_shadow.py:363-394` (`induction_goal_pool`: min_shared, one direction per pair, nwdef alias and placeholder exclusions). Flags `shadow_induction` (false), `shadow_induction_min` (3), `wmtm/runtime_config.py:155-156`. [2] Engine behaviour, fold formula ($\Sigma w \cdot s / \Sigma w$, $w = m \cdot c_a \cdot c_b$), ~0.001 per instance, base-rate path (0.016), measurements (0 at 3; one UNCLEAR rule at 2, conf 0.0024, count rule FAILS): `reports/t-2026-11-11-wmtm-induction/REPORT.md`. [3] Design: §9.1b.

12. PLN and Omega-PLN: what exists and what does not

This is not a component in the code. It is a question about the code: how much of PLN, and of Omega-PLN, is actually in wmtm? The seven-part layout is kept, but "worked example" shows a truth-value calculation rather than a pipeline step, and "inputs and outputs" describes where PLN appears.

12.1 Purpose

PLN (Probabilistic Logic Networks) is the logic the design assumes: atoms with (strength, confidence) truth values, and inference rules with formulas for how truth values combine. Omega-PLN is the newer extension of PLN, with richer, higher-order truth values and evidence that carries its provenance. You will want to know which of these wmtm actually exercises, so that

its conclusions are not read as more than they are.

12.2 Where PLN appears

- **Atoms.** The parser emits PLN-style atoms: Member, Inheritance, Implication, Evaluation, each with a truth value (section 4).
- **Inference.** PeTTaChainer is a PLN backward chainer. Its truth-value formulas come from a PLN formula library that ships with it. wmtm passes truth values in and stores what comes back (section 6).
- **wmtm itself** computes no truth values. Its own Python PLN engine was removed in October.

12.3 How it works: which inference patterns are real

Design §9.1 pattern	In the build
a) deduction	**Real.** Member-then-Inheritance, Inheritance chains up to 3 steps, and the parser's Implication rules applied to Member facts; Implication links are not chained (section 9).
b) induction	**Real but adds nothing yet** (section 11). Off by default.
c) abduction	**Not present.**
d) analogy	**Not present.**
e) contradiction handling	**Not present.** Conflicting items sit side by side.
evidence aggregation	**Not present** as a wmtm step. Two proofs of the same conclusion are not merged. The engine does revise evidence *within* one proof.

12.4 Worked example

Take the deduction from section 9. PeTTaChainer combines (Member mira falcon_trainer) at, say, (1.0, 0.9) with (Inheritance falcon_trainer lantern_keeper) at (0.95, 0.9). The strength of the conclusion is driven by the strength of the link, and its confidence is lower than either premise's, because confidence is lost at every step. A two-step chain loses more. The exact figures depend on PeTTaChainer's deduction formula, and on whether class base rates are available (wmtm supplies none). What wmtm shows the bot is just the final pair.

What Omega-PLN would add, and wmtm does *not* have: the conclusion would carry a richer truth value (for example a distribution, not just two numbers), and its evidence would record *which* notes it rests on, inside the truth value. Then two proofs that share a note could be recognised as not independent, and would not count twice. In wmtm today the source list (section 6) records the notes, but the truth-value arithmetic does not use it.

12.5 Differences from the design doc, and why

The design assumed a WMTM-internal PLN engine (§3.4) with the five patterns of §9.1. The build moved all PLN inference to PeTTaChainer, for the reasons in section 9.5, and so it inherits exactly PeTTaChainer's coverage. Omega-PLN is not mentioned in the design either, so its absence is not a deviation. It is simply not there yet.

12.6 Status

- PLN deduction through PeTTaChainer: on main, in use.
- Induction: on main, off.
- Abduction, analogy, contradiction handling, aggregation: open. They need rules upstream in PeTTaChainer (Roman's side) or goal generation in wmtm, or both.
- **Omega-PLN: not used anywhere in wmtm.** A search over the code, docs and reports finds no mention. I understand from the task order that its reference interpreter does not exist yet **[unverified: taken from the order]**.

12.7 Limits

- wmtm's conclusions carry only plain (strength, confidence) pairs.
- Evidence is not tracked inside the truth value, so independence is not checked.
- Coverage of PLN is set by PeTTaChainer's rule set at its pinned version, not by wmtm.

Sources. [1] Python PLN removed: design doc header note (Track L, 16 Oct); `docs/RUNTIME.md` "Reasoner switch". [2] Pattern table: `docs/overview/WMTM_OVERVIEW.md` §1.5 table; design-gap-check GAP-9; t-2026-11-11-wmtm-induction. [3] Engine formula library (`third_party/pln_formulas`) and base rates: `reports/t-2026-11-11-wmtm-induction/REPORT.md`. [4] Omega-PLN: grep for "omega" + "pln" over `wmtm/`, `docs/`, `reports/` at e59ab57, no match (re-checked by review 1 of the overview). Reference interpreter: MeTTaDinC OQ 6 via the order **[unverified]**. [5] Design: §3.4, §9.1.

13. The shadow/live split and the switches

13.1 Purpose

wmtm runs inside a bot that people talk to, so new machinery has to be tried without putting the live bot at risk. The *shadow/live split* is how that is done. Anything expensive or experimental runs as a background "shadow" process. It writes only to its own files and cannot change what the bot sees, unless a separate switch lets its results through and the results pass their gates. The *switches* are a small JSON file in the bot's memory folder. They turn features on and off without a restart. This component is not a box in the data-flow diagram. It decides which boxes are active.

13.2 Inputs and outputs

- **In:** the flags file (re-read every cycle); environment variables, which override it (for development and tests).
- **Out:** one resolved setting per feature for this cycle; background worker processes launched or not; a record of which settings came from where and which values were invalid.

13.3 How it works

1. **Read once per cycle.** At the start of each cycle wmtm reads the flags file and resolves every setting: the environment wins, then the file, then the built-in default. A missing or broken file means *everything optional is off*. An invalid value for a key is ignored and reported.
2. **Defaults are off.** The parse feed, the shadow worker, the goal engine, the goal suggestions, writeback, context parsing, speaker linking and induction are all off unless switched on.
3. **The reasoner setting is special.** It defaults to PeTTaChainer, and in that mode the shadow worker is launched even if the shadow switch itself is off, because admission needs its output. Setting the reasoner to anything else, including the old value for the removed Python engine, turns reasoning off: nothing is admitted on any path.
4. **Background, never blocking.** Workers are detached processes at the lowest CPU and I/O priority, with a memory cap. Because some host schedulers reset priorities, the worker re-asserts them every second. The turn never waits for a worker, and every worker error is swallowed on the turn's side.
5. **Two kinds of turn.** On a *real* turn (a person wrote) the full cycle runs. On an *automated* turn (the bot's own loop, nobody writing) a fast path runs instead. It does not load the full engine. It runs the same forgetting rule and the same hooks (queue for parsing, maybe launch the shadow, admit from a fresh admit file) in a few milliseconds. Shadow launches are limited to one every 5 minutes on both paths, so idle ticks cannot spawn a stream of runs.
6. **Results cross only through gates.** The only route from the shadow into the store is the admit file and its checks (section 7). The only route from the goal engine to the bot is the suggestion block, behind its own switch (section 10).

13.4 Worked example

Suppose the bot's flags file contains only `{"semparse_feed": true}`. Then:

- Notes are queued and parsed in the background (the feed is on).

- The reasoner setting is not in the file, so it is PeTTaChainer. The shadow worker therefore runs every 5 minutes or so, and fresh, gated conclusions such as "mira is a lantern_keeper" are admitted.
- The goal engine is off, so no proposals are made and Mira never sees a suggestion line.
- Writeback is off, so "mira is a lantern_keeper" never reaches LTM.

If an operator adds `"reasoner": "off"`, then from the next cycle on nothing is admitted, without a restart. Parsing continues, since the feed has its own switch.

13.5 Differences from the design doc, and why

The design has no shadow processes and no switches. Its inference ran inside the cycle. The split exists because the reasoner is too slow to sit inside a turn (section 9.3), and because every feature after the first was trialled on the live bot. A file re-read every cycle lets an operator back a feature out in seconds.

13.6 Status

On main. The setting resolver lists every key with its default and its environment name. Which keys the live bot's flags file actually sets I cannot see from my sandbox **[unverified: live flags]**. The task record says the live bot parses in legacy mode and that the vendored runtime there predates the newer parse-mode key **[unverified]**.

13.7 Limits

- The behaviour of the live bot depends on a file outside the repository, so the code alone cannot say what Omega runs. Every claim about "live" in this guide is about defaults unless marked.
- The rule that the PeTTaChainer reasoner launches the shadow even with the shadow switch off is easy to overlook. Turning reasoning off takes the reasoner key, not the shadow key.
- Background work is invisible by design. A worker that silently stops producing results only shows up as "nothing admitted", with a skip reason in the admit log.

Sources. [1] Resolver, env > file > default, invalid keys: `wmtm/runtime_config.py` (SPEC, `resolve`); `docs/RUNTIME.md` "Enabling: the flags file". [2] Reasoner switch and forced shadow launch: `docs/RUNTIME.md` "Reasoner switch (v0.5)"; `wmtm/orchestrator.py` `cycle` step 2b' (`force=True` when the mode is pettachainer). [3] Worker priorities: flags `worker_mem_mb` (3072), `worker_nice` (19), `worker_sched_idle` (true), `worker_prio_refresh_s` (1). [4] Fast path: `wmtm/context.py:558` (`fast_path_automated`); flag `hooks_on_automated` (true); `shadow_min_interval_s` (300). [5] Live parse mode: t-2026-11-10-wmtm-flag-semparse-legacy, t-2026-11-10-wmtm-gateway-callers (order attachment).

14. The parse budget

14.1 Purpose

Every parse is an LLM call that costs money, and the bot and several hive tasks share the same provider. The parse budget makes sure that the live bot always has room to parse, that experiments cannot starve it, and that nobody re-pays for a parse already done. It sits on the arrow from the parse feed to the LLM.

14.2 Inputs and outputs

- **In:** each caller's role (*live* or *task*); a ledger of recent calls shared by every process on the host; the parse cache.
- **Out:** permission to make one call or not; one ledger row per call; when the share is used up, a stop with the status "share full".

14.3 How it works

1. **Never parse twice.** The cache is keyed by a hash of the note's content. A note already in the cache costs nothing. Changing the prompt does not re-parse old notes automatically. That has to be done on purpose, one call each.

2. **An overall cap.** At most 20 parse attempts per rolling hour per feed.
3. **Fixed shares.** On top of the cap, the live bot's feed gets 10 calls per rolling hour, counted across all its variants (base, context, speaker). All hive tasks *together* get another 10, counted in a ledger shared across the host. A process has to declare itself live to use the live share, and anything else counts as a task.
4. **Write before you call.** Each call writes its ledger row *before* the request is sent. A crash after sending therefore still counts, and two processes cannot both slip in the last slot.
5. **Tasks read, never write, live data.** A task may read the live bot's parse files but not change them.
6. **Prompt caching.** Most of the parser prompt is the same for every note, so the provider's prompt cache should make repeated calls cheap. It was found not to work, because the bot's OpenClaw gateway put per-request text in front of the cache boundary. This was a gateway problem, not a wmtm one. A patch was applied on the host, with a daily check that it is still in place **[unverified: host]**.

14.4 Worked example

At 10:00 the live feed has used 9 calls in the last hour, and two experiments have used 10 between them. Notes A and B are queued. Note A is already in the cache, so it costs nothing. Note B needs a call. The live share has one slot left, so a ledger row is written and the call goes out. The live share is now full. An experiment that tries to parse at 10:01 finds the task share full and stops with "share full". The measurement tool waits by default rather than failing. At 10:40 the oldest live calls fall out of the rolling hour and the live feed can parse again.

14.5 Differences from the design doc, and why

The design has no LLM in the loop (its parser was meant to be a simple extractor), so it has no budget. The budget exists because of the parser deviation in section 4. The fixed live/task shares were andy's decision of 10 November, after experiments competed with the live bot for calls.

14.6 Status

On main and always active for the feed. Shares and ledger since 10 November. The gateway patch and its daily guard live on the host, not in this repository **[unverified]**.

14.7 Limits

- Ten calls an hour is a small number. A burst of new notes takes hours to parse, and the reasoner works on the old parses meanwhile.
- The shares count calls, not tokens, so a long note costs the same slot as a short one.
- The cache keys on exact content. A note edited by one character is a new note and is parsed again.

Sources. [1] Cache key, cap, shares, ledger-before-send, `share_full`, role: `docs/RUNTIME.md` (Track H paragraph); `wmtm/parse_budget.py`; flags `parse_cap_per_hour` (20), `parse_role`, `parse_share_live_per_hour` (10), `parse_share_tasks_per_hour` (10), `parse_live_dir`, `parse_budget_dir`. [2] Shares decided by andy 2026-11-10; t-2026-11-10-wmtm-shared-parse-count-v2. [3] Prompt caching: t-2026-11-10-wmtm-parse-caching, t-2026-11-10-wmtm-gateway-patch-pack, t-2026-11-11-wmtm-gateway-patch-guard (order attachment).

15. Evaluation: how wmtm is judged

This section is about the judging rather than a component. It keeps the seven-part layout: "inputs and outputs" are what an evaluation takes and produces, and each scoring rule gets a small synthetic example.

15.1 Purpose

Two questions need different tests. *Does a change make things worse?* That is a no-regression question, and it is answered by A/B replays and the count rule. *Does wmtm actually work?* andy

decided on 10 November that this is judged by the design doc's §12 tests and by Mesto's worked examples, not by the count rule.

15.2 Inputs and outputs

- **In:** a fixed set of about 20 evaluation notes from one bot; their *frozen parses* (the parser output saved once and reused, so both arms see exactly the same atoms); the live configuration (A) and a candidate (B); the written scoring rules.
- **Out:** for each arm, a list of conclusions, each labelled correct (C), wrong (W) or unclear (U); a per-conclusion diff between the arms; a pass or fail under the count rule; a recommendation (deploy or not), which andy decides on.

15.3 How it works

1. **Frozen parses.** The parser does not give the same output twice: in one check, all 20 notes came out differently across three fresh runs, and the atom totals varied by about 10 %. So both arms replay the same saved parse. A difference between A and B is then a difference in reasoning, not parser noise.
2. **A and B.** A is how the live bot behaves; on the evaluation notes it gives 2 correct and 2 wrong conclusions. B is the candidate change.
3. **The count rule.** B passes only if it has at least as many correct conclusions as A *and* no more wrong ones. Unclear ones are reported but do not count. With a baseline of 2 wrong, a single extra wrong conclusion fails B.
4. **The per-conclusion diff (Mesto §11).** Every conclusion is listed by id as kept, gained or dropped between A and B. Each drop is classified as a real loss or a harmless one. Counts alone can hide a swap of a good conclusion for a bad one; the diff cannot.
5. **W1, cross-note conflation.** A conclusion counts as wrong when its premises come from more than one note *and* one of them is a placeholder fact. Placeholders are local to a note: "a dog" in one note and "a dog" in another need not be the same dog. One exception: an identical class-level fact found in two notes is corroboration, not conflation, and is not W1.
6. **Restatements are counted apart.** A conclusion can be correct and still useless. Restatements (by head noun, by symmetry, by coreference) are listed in their own column.
7. **The reader pass.** A reader lists what a person would conclude from the notes. B is scored on how many of those it reaches. This measures useful reach, which the count rule cannot.
8. **Process.** Every report gives its pins (parser, PeTTa, PeTTaChainer, rule-file checksum), checksums of its evidence and a script that recomputes its tables. A reviewer re-runs the key numbers before the task closes. Reports contain ids, hashes, counts and rule shapes, never note text.

15.4 Worked examples

Count rule. Suppose A, on the running example, derives "Mira keeps the lantern log" (C) and one wrong conclusion. B adds "Mira is a trainer" (C, but a restatement) and a new wrong one. B has 2C/2W against A's 1C/1W: more correct, but also more wrong, so B **fails**. If instead B's extra conclusion had been right, B would pass, even though its only gain is a restatement. That is why the count rule is only a guard.

Per-conclusion diff. Suppose B passes the counts, 1C/1W against 1C/1W, but has dropped "Mira keeps the lantern log" and gained "Mira is a trainer". The counts are equal. The diff shows one real loss.

W1. Note A says "Mira trains falcons" and the parser invents a placeholder `sk_1` for "the falcons". Note C, about a market, also mentions falcons, and the parser invents its own `sk_1`-like placeholder there. A chain that joins the two placeholders, for example to conclude that Mira's falcons are for sale, uses premises from two notes and a placeholder: W1, wrong. But if Note B and Note D *both* say "falcon trainers keep the log", a conclusion using that fact from either is not W1: the same class-level fact, stated twice, is corroboration.

Reader pass. A reader of Notes A and B would expect "Mira keeps the lantern log this week". If B derives exactly that, it scores 1 of 1, whatever else it derives.

15.5 Differences from the design doc, and why

The design's own test plan is its §12: unit tests for the store, attention, forgetting and inference. Those were turned into an automated test file, which now gives 19 passes and 5 expected failures, each failure naming the gap it waits on (section 16). The A/B replays, the count rule, W1, the restatement column and the reader pass are not in the design. They grew out of the first replays, when counts looked good but the conclusions were mostly restatements.

15.6 Status: what the numbers say

- **Baseline:** A = 2C/2W.
- **No wmtm-side rules** (November 8): B 8C/2W/1U. The count rule holds. Deploy: not yet.
- **Scoring 49 goals** (legacy parse): 41C/8W. 39 of the 41 correct ones were one-step "an X_Y is a Y" restatements. Fails on wrongs. This is where the trivial filter came from.
- **Narrowed-class pre-load** (three frozen parses): goals went from 0 to 12, 14 and 10, all proved; 6C/0W, 6C/2W, 5C/0W. Holds on the judged subset.
- **Induction:** nothing new at the default threshold (section 11).
- **Sentence-by-sentence parsing:** 14 conclusions, all trivial echoes, 0 wrong.
- **Latest parser with 280 seeded rules** (November 12): 105 of 105 goals proved, 97C/4W/4U. 82 of the 105 are restatements (77 by symmetry, 5 by coreference), and the trivial filter catches none of them. Fails as a whole (4 wrong against 2). Holds on the 23 non-restatements (22C/1W). The per-conclusion diff shows 10 dropped, 4 of them real losses. The reader pass: 75 expected conclusions, 8 reached. Recommendation: do not adopt yet.

In plain words: wmtm's reasoning is mostly *correct*, but most of what it proves is not *new*, and it reaches only about a tenth of what a reader would conclude, chiefly because the parser does not emit the needed premises and some needed rules do not exist.

15.7 Limits

- About 20 notes from one bot. Small, and not independent.
- Labels are judged by people or agents under written rules. They are reviewed, but not double-blind.
- Offline only. No study measures what users of the live bot actually experience.
- The goal engine has never been in an A/B.
- Every result holds at its pins. A new parser or PeTTaChainer version means re-running.

Sources. [1] "Working" = §12 tests + Mesto's examples; count rule = guard; W1 and its corroboration exception: order attachment `SCORING-RULES.md` (andy 2026-11-10; Work msgs 285, 355). [2] Parse variance: t-2026-11-10-wmtm-parse-variance (20/20 notes differ; 810/896/842 atoms). Frozen parses: `wmtm/frozen_parse.py`. [3] Count rule pre-registered: t-2026-11-07-wmtm-f3-ab item 1. Baseline A_v3 2C/2W: t-2026-11-08-wmtm-f3-ab-v3. [4] §12 tests: `tests/test_design_s12_gapcheck.py` (19 passed, 5 xfailed at e59ab57), `tests/test_design_s12_engine.py`. [5] Results: t-2026-11-08-wmtm-f3-ab-v3, t-2026-11-10-wmtm-score-49-goals, t-2026-11-10-wmtm-narrow-preload, t-2026-11-11-wmtm-induction, t-2026-11-12-wmtm-sentence-reasoning, t-2026-11-12-wmtm-parser-master-adopt-v2 (order attachment `wmtm-task-outcomes.md`; `docs/overview/WMTM_OVERVIEW.md` §2, §3.2).

16. Status and next steps

16.1 What is built and on main

Component	On main	Default	Measured?
Intake and parser (legacy mode, lowering, quality gate, cleanup)	yes	parse feed off	yes: parse variance, coverage, sentence mode

Component	On main	Default	Measured?
Store and items	yes	always on	§12 tests pass
Truth values and sources	yes	on with the reasoner	exact values in tests
Attention, admission, trivial filter, forgetting	yes	on (writeback off)	§12 tests; trivial filter on three frozen parses
Recall, graph recall, context block	yes	on	graph-only and seeding probes
PeTTaChainer reasoner (shadow worker)	yes	reasoner on; runs once parses exist	A/B replays (section 15)
Goal ranking	yes	on with the shadow	indirectly, through the A/B replays
Goal engine and suggestions	yes	off	**never in an A/B**
Goal lifecycle	yes	offline tool	adversarial test set
Induction	yes	off	adds nothing yet
Switches and shadow/live split	yes	file re-read every cycle	split probe
Parse budget	yes	on	ledger checks

The live bot runs an older vendored release (rc2), so not everything in this table is live there **[unverified: current live release]**.

16.2 What is open, and who decides

Open item	What it would change	Who
The parser emits few class-level links	The main limit on what the reasoner can derive (sections 4, 9)	Man Hin, or a wmtm-side fallback; andy decides
Missing rules: ordering of times, supersession, sums, contradiction	34 of 75 expected conclusions need them	Roman (PeTTaChainer) and Man Hin
Adopting the latest parser with seeded rules	105 proved, but mostly restatements; fails the count rule	andy decides; not recommended yet
Extending the trivial filter to symmetry and coreference restatements	Would remove most of the 82 restatements	andy decides
A real useful-vs-trivial signal	Utility still credits display, not use (section 7)	ours
Writing durable conclusions back to LTM	Off until the signal above exists	Ben and andy decide
Graph-recall seeding default	Churn against graph-only reach (section 8)	andy decides
Composite eviction, contradiction handling, writeback in the design's form, forgotten notes in LTM, LTI in recall	The design's §12 expected failures	ours
Abduction, analogy, evidence aggregation	Inference breadth (section 12)	Roman plus ours; Ben decides whether still in scope
Evaluating the goal engine	Its value is unknown	ours
Omega-PLN	Richer truth values, provenance inside the truth value	upstream; reference interpreter not yet available **[unverified]**

16.3 Temporary workarounds to remove later

- The narrowed-class rewrite before loading (section 4). Remove when PeTTaChainer accepts narrowed classes inside statements.
- Equivalence statements not passed to PeTTaChainer. Remove when PeTTaChainer accepts them.

- The gateway prompt-cache patch on the host (section 14). Remove when OpenClaw fixes it upstream.
- The seeded rule file of 280 rules on the parser-adoption branch. Remove when the parser emits those rules itself.

16.4 Suggested next steps, in order

1. **Get class-level links out of the parser.** Every measurement points here. Without premises like "falcon trainers keep the lantern log", the reasoner can only restate notes. This is a parser task, with wmtm supplying the evaluation notes and the reader pass as the yardstick.
2. **Make the trivial filter catch symmetry and coreference restatements.** Cheap, and it makes every later count honest.
3. **Score on the reader pass, not just the count rule.** 8 of 75 is the number that should move.
4. **Then decide on writeback**, once a useful-vs-trivial signal exists.
5. **Run the goal engine in one A/B**, with its suggestions on, before anyone relies on it.
6. **Decide on inference breadth.** If abduction, analogy and contradiction handling are still wanted, they need rules in PeTTaChainer first.

Sources. [1] Status table: sections 4-15 and their sources; `docs/overview/WMTM_OVERVIEW.md` §3.1-3.4. [2] Open items and deciders: overview §3.3; t-2026-11-10-wmtm-design-gap-check (GAP-3 to GAP-10); t-2026-11-12-wmtm-parser-master-adopt-v2; t-2026-11-11-wmtm-spread-seeds-probe; f3-ab-v3 criterion 5. [3] Stopgaps: overview §3.4; `docs/STOPGAPS.md` (PC-EQ1); `semparse_adapter.py` (STOPGAP narrow_preload). [4] Live release rc2 4d2ab315: t-2026-11-10-wmtm-gateway-callers. [5] Next steps are this guide's suggestion, not a decision.

Appendix C. One cycle, step by step

Sections 4 to 14 describe the components one at a time. This appendix puts them back in the order the code runs them, once per turn. It is the build's version of the design's §4, "The WMTM cycle".

```
turn starts: load saved store
      |
      v
1   recall from LTM (keyword + graph, budget 10)
1b  goal-context boost (STI up for items matching the active goal)
2a  read the flags file once for this cycle
2b  reasoner phase: admit gated conclusions from the last shadow run
2b' launch background workers: shadow worker (always, in PeTTaChainer mode), parse feed (if on)
2c  GoalChainer feedback phase (only for a configured goal request)
4-6 tick, utility, reinforce used items, forgetting policy  -> forgetting log
7   writeback candidates to LTM (derived: off by default)
8   goal lifecycle check (expire stale goals)
      |
      v
build context block; save store; turn ends
```

1. Recall. Notes come in from LTM as in section 8. Each gets a starting STI from its recall score, and its origin time is the journal entry's time, not the time of recall.

1b. Goal-context boost. If the orchestrator holds an *active goal*, in the design's sense of a task context, every item that shares longer words with that goal's description gets an STI boost, up to 2.0 when every keyword matches. This is the design's §8 goal context. It is separate from the goal engine of section 10. Most turns have no active goal, so this step usually does nothing.

2a. Flags. The flags file is read once, so every later step in the cycle sees the same switches (section 13). A scratch orchestrator, one without a state folder, always runs with the reasoner off.

2b. Reasoner phase. Admission reads the shadow worker's admit file and applies the checks of section 7. On a turn where the cycle itself is skipped for time, and on automated turns, admission still runs on a shorter path, so a conclusion is never lost just because the turn was busy.

2b'. Background launches. The shadow worker is started whenever the reasoner is PeTTaChainer (the default), even if the shadow switch is off (section 13). The parse feed is started only if switched on. Neither is waited for. Neither can raise an error into the turn.

2c. GoalChainer feedback phase. A narrower, older hook: if the orchestrator was given a GoalChainer request, the decisions come back as derived items. It needs its own configuration and is not used by the live bot **[unverified]**. It is not the log-only goal engine of section 10.

4-6. Forget phase. One function does this, and the automated fast path calls the same function. The store ticks (decay, ageing, the 0.01 floor), utility ticks, items used this turn are reinforced, and the forgetting policy runs. Every eviction goes into the forgetting log. Wall-clock stale cleanup runs through a sibling function that also logs every eviction.

7. Writeback. Items over the writeback thresholds become LTM notes. Derived items from PeTTaChainer are excluded unless that switch is on (section 7).

8. Goal lifecycle check. Active task-context goals whose time has run out are marked abandoned.

Running example in one cycle. On the turn of the question, step 1 brings in Notes A and B. Step 2b finds no conclusion about Mira yet, because the notes have not been parsed. Step 2b' queues both notes and launches the shadow worker, which has nothing about Mira to work on yet. Steps 4-6 evict one old item. On a later turn, after the parse feed has run and the shadow worker has run again, step 2b admits "mira is a lantern_keeper".

Sources. `wmtm/orchestrator.py:445-559` (`cycle`, `forget_phase`, `stale_cleanup`), `:235-250` (`_goal_context_boost`, 2.0 × overlap / keywords), `:390-415` (`_run_goalchainer_phase`); budget-skipped and automated admission: `docs/RUNTIME.md` "Reasoner switch (v0.5)"; design §4.

Appendix D. The running example: the state after each step

Step	Where	What exists now	Turn
0	LTM	Notes A, B with an evidence-for link	before
1	store	request item (user); B (recalled, keyword); A (recalled, graph link)	1
2	context block	B and A near the top; no derived line yet	1
3	parse queue	hashes of A and B queued	1
4	parse cache	A: Member mira falcon_trainer; B: Inheritance falcon_trainer lantern_keeper	when budget allows
5	shadow KB	the two atoms, the defining rule, the canary	next shadow run
6	shadow goals	Member mira lantern_keeper (ranked first); Member mira trainer	same run
7	admit file	both proved; canary passed	same run
8	store	"mira is a lantern_keeper" admitted (derived, from A and B); "mira is a trainer" refused as trivial	next turn
9	goal log	ClarifyQuestion to Mira about the gatehouse (if the engine is on)	next shadow run
10	context block	derived line with its tv and sources; a suggestion line only if Mira is speaking and suggestions are on	next turn

Step	Where	What exists now	Turn
11	store	STI decays; utility rises only if the line keeps being shown and used	following turns
12	forgetting log	conclusion evicted; derivation key (text + A, B) recorded	hours later
13	store	the same conclusion from A and B is dropped as known; from new notes C and D it is admitted	later